# FP64 Emulation in cuBLAS on Consumer GPUs: Analysing What a Single Switch Delivers

Taidi Zhou
*Independent Undergraduate Research Project*
*November 2026*

***Abstract*—GPU makers are allocating ever more die area to low-precision matrix units, and NVIDIA's cuBLAS library now offers to compensate by emulating double-precision (FP64) matrix multiplication on integer tensor cores, enabled by a single environment variable. We measure what this switch delivers on three consumer GPUs, the RTX 3090, RTX 4090, and RTX 5060 Ti, across square and non-square shapes, inputs with wide value ranges, four library releases, and a randomised singular value decomposition. With one extra memory setting in the code, plus a second environment variable on the RTX 3090, emulation ran 14.5 to 18.8 times faster than native FP64 on square products at its peak and was at least ten times more accurate. The switch alone delivered much less. The library's default strategy never emulated on the RTX 3090, and on the RTX 4090, with a synchronisation after every call, the switch reached as little as 8 % of the speed obtained with that setting and made some products slower than native, because the working memory that each emulated call allocated was given back at every synchronisation, contrary to the library's release notes. When the values summed in a dot product spanned too wide a range, the library fell back to native arithmetic without any error or warning, at the same point on all three GPUs. In the decomposition, emulation was up to three times faster at FP64 accuracy, though slower at the smallest sketch sizes. Emulation was thus accurate by default in our tests, but its speed has to be checked on each GPU.**

***Index Terms*—cuBLAS, double precision, floating-point emulation, GPU, matrix multiplication, mixed precision, Ozaki scheme, performance evaluation**

## I. Introduction

GPU makers are allocating ever more of their die area to low-precision matrix units, and double precision (FP64) is losing ground. On NVIDIA's datacentre GPUs, native FP64 throughput is stagnating while emulated FP64 grows; HPCwire, citing NVIDIA, reports that the upcoming Rubin GPU will deliver 33 TFLOPS of native FP64 matrix throughput but 200 TFLOPS with emulation [1]. On consumer GPUs the imbalance is extreme, as the INT8 tensor cores of an RTX 4090 offer nearly 512 times its FP64 throughput [2]. Emulation turns this imbalance into an opportunity. The Ozaki schemes rewrite an FP64 matrix product as exact products of low-precision numbers that integer units can compute [3], [4], and Matsuoka goes as far as to argue that FP8 arithmetic with the second Ozaki scheme (Ozaki-II) can replace FP64 hardware altogether [5].

Since CUDA 13.0 Update 2, cuBLAS can emulate FP64 matrix multiplication on integer tensor cores [6], guarded by an automatic dynamic precision (ADP) framework that chooses the precision for each call and runs native FP64 instead when emulation would be inaccurate or slow [7]. A user enables it by setting a single environment variable, without any change to the code, and the choice then extends, where applicable, to the libraries that call cuBLAS [8], [9]. Moreover, NVIDIA has announced its intention to switch the feature from opt-in to opt-out "after enough exposure" [9], at which point every FP64 matrix multiplication sent to cuBLAS would become a candidate for emulation by default.

Whether this switch delivers what it promises is, however, disputed. AMD's Nick Malaya argued, as reported by HPCwire, that performance drops below native FP64 when a workload does not have square matrices and that emulation has "accuracy problems" when matrix elements "differ by a few orders of magnitude" [10], and he told The Register that it uses "about twice the memory capacity" of native FP64 [11]. The published evidence does not settle the question for most users. NVIDIA's own evaluation uses square products on top-end Blackwell GPUs [7], academic studies measure their own implementations [4], [12], and the one independent performance comparison with cuBLAS disables its automatic precision and tests square matrices on datacentre GPUs only [13]. For consumer GPUs, where native FP64 is weakest and emulation should help most, we found only informal benchmarks of square products [14], [15].

To our knowledge, no study has yet measured what cuBLAS's shipped default delivers to a user who simply turns it on. We therefore treat the library as a black box and observe it as a user would meet it. On three consumer GPUs, one each from the Ampere, Ada Lovelace, and Blackwell generations, we sweep square and non-square shapes, inputs with widening exponent ranges and special values, four library releases, and the memory settings a user might or might not change, and we finish with an end-to-end randomised singular value decomposition (SVD). For every call, we record its time, its accuracy, and whether cuBLAS emulated it, declined to, or fell back during the call.

The capability turns out to be real, but the switch alone does not always deliver it. With a single extra memory setting in the code, which we call the one-line fix, emulation ran 14.5× to 18.8× faster than native DGEMM, cuBLAS's FP64 general matrix multiplication (GEMM), on square products at its best, and was at least ten times more accurate. On the RTX 3090, however, this also took a second environment variable, one that selects cuBLAS's EAGER strategy and so makes it emulate

wherever it can, because the default strategy never emulated there. With the switch alone, which we call the zero-code path, and a synchronisation after every call, the RTX 4090 reached as little as 8 % of the speed of the one-line fix and ran some products more slowly than native, because every emulated call allocates its workspace anew and the memory is given back at each synchronisation, contrary to what the release notes of this cuBLAS version describe. Calls that run back to back avoid almost all of this cost. When the values summed in a dot product spanned too wide a range, so that emulation would have needed more than about 107 mantissa bits, cuBLAS fell back to native FP64 at the same point on all three GPUs and without any error or warning. In the randomised SVD, the one-line fix with EAGER made the decomposition at best 2.25× to 3.04× faster than with native FP64, at the accuracy of FP64, but slower than native when the sketch was small.

**Contributions.** This paper makes four contributions:

1) a measurement study of cuBLAS's automatic FP64 emulation, as shipped in cuBLAS 13.8.0.4, on three consumer GPU architectures, covering square and non-square shapes, wide exponent ranges and special values, four library releases, and an end-to-end workload;
2) a characterisation of the two ways in which a call can end on native FP64, namely the default strategy's decision not to emulate, which depends on the GPU, and the run-time fallback, which sets in at the same precision on all three GPUs and is silent, together with evidence that none of the 142 configurations the default emulated, where we also measured native, ran slower than native in GPU time;
3) an explanation of the cost of the zero-code path, through a model of per-call allocation that predicts the measured penalty when a program synchronises after every call, evidence that the library behaves differently from its documentation, and a demonstration that either the workspace the documentation recommends or a single setting of the memory pool removes the penalty, the latter also through CuPy, where a workspace does not take effect; and
4) an assessment of accuracy, in which emulation with dynamic precision was never less accurate than native FP64 in any of 2624 measurements of single products, stayed at the level of FP64 in the final answers of the workload, produced identical results on all three GPUs, and, in our experiments, lost accuracy only when the user fixed the precision too low.

**Organisation.** Section II reviews related work and Section III the background on the Ozaki schemes and their implementation in cuBLAS. Section IV formulates the problem and our hypotheses, and Section V describes the methodology. Section VI presents the results, from single products to the workload, and Section VII concludes with the limitations of this study and directions for future work.

## II. Related Work

Work on FP64 emulation falls into four strands: the Ozaki schemes and their implementations (II-A), their integration into vendor libraries (II-B), independent evaluations of those libraries (II-C), and applications (II-D). We close each strand with how our study differs from it.

### A. Ozaki-scheme emulation

Most work on FP64 emulation develops or accelerates an Ozaki scheme and measures its own implementation. After Ozaki et al. introduced the error-free splitting of a matrix product [3], Ootomo et al. carried it to the integer tensor cores of GPUs and computed DGEMM faster than cuBLAS on consumer cards [16], and Uchino et al. improved the performance of this INT8 approach further [2]. Ozaki Scheme II then replaced splitting with modular arithmetic and reached 7.4 to 9.8 TFLOPS of FP64-equivalent throughput on an RTX 4090, above the native rate its authors measured [4]. Uchino et al. later reported a 1.4× speedup over native DGEMM on a GH200 with this scheme, which they also evaluated on an A100 and on a consumer RTX 5080, for square products [12]. Further variants move the schemes onto FP8 tensor cores [17], [18] or improve the scaling of Ozaki-II's fast mode [19]. Alongside these speed studies, error analyses show that the precision required depends on the inputs. Abdelfattah et al. find that the integer scheme "may require an extremely large number of slices if the matrices are badly scaled" [20], and Uchino et al. show that a wide distribution of exponents raises the number of moduli that Ozaki-II needs [21]. Our study builds on this line but asks a different question. Rather than proposing or tuning an implementation, we measure what the one shipped in cuBLAS does when a user switches it on, including when it chooses not to run at all.

### B. Emulation in vendor libraries

NVIDIA brought emulation into cuBLAS with the ADP framework of Schwarz et al., whose safeguards they report to cost under 10 % while reaching up to 2.3× over native DGEMM on a GB200 and 13.2× on an RTX Pro 6000 Blackwell Server Edition, in a 55-bit setting [7]. Their implementation builds on Ozaki-I, leaves extensions to Ozaki-II "beyond the scope of this study", and is evaluated on square products. NVIDIA's blog presents results for the same two classes of GPU, together with a 1.5× end-to-end speedup of Quantum ESPRESSO on the RTX Pro 6000 [6]. Outside NVIDIA, RIKEN's GEMMul8 library provides Ozaki-II with INT8 and FP8 back ends and a hook mode that intercepts cuBLAS calls without code changes [22]. The vendor evidence thus comes from top-end Blackwell parts and square products. We instead evaluate the shipped default on three consumer architectures and across shapes, in the Ozaki-II pipeline of cuBLAS 13.8.0.4, which the ADP paper did not cover.

### C. Independent evaluations

Independent studies of cuBLAS's emulation are few, and they configure it differently from a default user. Lu et al.

present EmuGEMM, fused integer tensor-core kernels for both Ozaki schemes, and compare them with cuBLAS's Ozaki-I emulation, for which they disable ADP and select the slice count explicitly, using CUDA 13.1 on GH200 and B200 GPUs [13]. They also note that Ozaki implementations "underperform high-precision TCs [tensor cores] on small matrices because preprocessing overhead dominates", and they state that they "do not test non-square or irregularly-sized matrices". Demmel et al., who grade the accuracy of Basic Linear Algebra Subprograms (BLAS) implementations, ran cuBLAS under its EAGER strategy (Section III-B) on one B200 and observed that "even when the strategy was set to EAGER, it used the FP64 DGEMM path" for their wide-range test, but they leave "tests on the reliability of these techniques" outside their scope [23]. On consumer GPUs, the only results we found are informal and self-published. A benchmark on an RTX 5090 reports 8.22× and 5.81× for square DGEMM on narrow- and wide-range inputs under the default path [14], and one on an RTX 5080 notes that the default strategy chose native FP64 at $N = 512$, where EAGER with Ozaki-II was 2.4× faster [15]. Our study covers what these leave open, namely the default strategy and the run-time fallback as a user meets them, on consumer GPUs and for non-square shapes, together with the memory cost of the zero-code path.

### D. Applications

Application studies have so far used emulation with a fixed precision or through other implementations, and on datacentre GPUs. Liu et al. offloaded the BLAS calls of the MuST application, without code changes, to the ozIMMU library on a GH200 and found that "accuracy depends on both arithmetic precision and the properties of the operator"; performance was not their focus, and the emulated run took 731.8 s against 412.1 s with native FP64 [24]. Their follow-up is the closest prior use of cuBLAS's own emulation in an application, running the EAGER strategy with fixed mantissa bits on a GB200 to study accuracy [25]. Luszczek et al. report that LU factorisation with INT8 split emulation needed at least 7 splits to pass the High-Performance Linpack (HPL) scaled-residual check on a test-matrix family designed to widen the numerical range of the entries, using ozIMMU on Hopper GPUs [26]. Randomised low-rank approximation, our workload, has been studied in reduced precision on tensor cores [27], [28]. Baboulin et al., for instance, report an average accuracy of order $10^{-4}$ for a standard single-precision (FP32) version and of order $10^{-2}$ for their faster mixed-precision approach, improved to order $10^{-5}$ with refinement. We instead require FP64 accuracy, run the decomposition under the default zero-code path, and trace which of its products emulate or fall back.

## III. Background and Preliminaries

This section summarises what was known before our study: how the Ozaki schemes turn an FP64 matrix product into exact products of cheaper numbers (III-A), how cuBLAS decides when to use them (III-B), and what a user must do to opt in (III-C).

### A. The Ozaki schemes

Emulation rests on an idea that Ozaki et al. introduced in 2012: a product of two floating-point matrices can be rewritten, without error, as a sum of products of split matrices, so that "the dominant part of our algorithm is constituted by ordinary floating-point matrix multiplications" [3]. Each input is split into slices of a few bits, chosen so that every product of two slices is exact in a lower-precision unit, and the partial products are then summed in higher precision. The price is the number of products, which grows quadratically; with $s$ slices per input, the scheme needs $s(s+1)/2$ of them [4]. Ootomo et al. [16] later stored the slices as INT8 values and relied on the 32-bit integer results of integer tensor cores to keep each product exact, and in this way computed DGEMM faster than cuBLAS on consumer GPUs. Such GPUs are where the idea pays most, since on the RTX 4090, as Uchino et al. note, INT8 tensor-core throughput is nearly 512 times that of FP64 [2].

Ozaki Scheme II removes the quadratic cost by replacing splitting with modular arithmetic. Ozaki, Uchino, and Imamura [4] scale the rows of $A$ and the columns of $B$ by powers of two and round them to integer matrices; for $s$ pairwise coprime moduli, they reduce both matrices modulo each modulus, compute one exact integer GEMM per modulus, and reconstruct the product from the residues with the Chinese Remainder Theorem. The number of GEMMs therefore grows only linearly with $s$. The precision needed, however, still depends on the inputs, and Uchino et al. show that a wide distribution of exponents calls for more moduli to reach the same accuracy [21].

### B. Emulation in cuBLAS

cuBLAS has shipped such an emulation since CUDA 13.0 Update 2 [6]. According to its documentation, the emulation "uses algorithms based on the Ozaki-I and Ozaki-II schemes" and runs on compute capabilities 8.x to 12.x, a range that includes all three of our GPUs [8], and from CUDA 13.4 onwards, the release notes add, cuBLAS uses Ozaki-II "when it provides a performance benefit over the Ozaki-I scheme" [29]. Before multiplying, cuBLAS converts each input to a fixed-point form, a shared power-of-two scaling factor times an $M$-bit integer mantissa, and the documentation is specific about what is shared: "elements in the same row of the A matrix, or column of the B matrix, share the same scaling factor, so the dynamic range within an A row or B column is encoded as part of the mantissa" [8]. A wide range of exponents along the summation index therefore costs mantissa bits, whereas a scale that is constant across a row of $A$ or a column of $B$ is absorbed by its scaling factor.

How many bits are enough is decided at run time by the ADP framework of Schwarz et al. [7]. Their workflow begins with a scan of $A$ and $B$, and "if either scan detects Inf or NaN, the workflow falls back to native FP64 GEMM". An estimator, the exponent span capacity, then determines how many mantissa bits FP64 accuracy requires, after which "a lightweight heuristic evaluates whether emulation is expected to outperform native FP64" and launches native FP64 GEMM

when it is not. All of these steps run as GPU kernels, so none of them needs a host synchronisation. In cuBLAS, ADP is the default behaviour, under the name of dynamic mantissa control, and the documentation states the consequence: when the required bits exceed a library default or a user-set maximum, "the framework dynamically dispatches to native FP64" [8]. That default, moreover, "is automatically chosen by cuBLAS and may vary based on problem size and GPU architecture". Users who prefer a predictable cost can instead fix the number of bits, which trades accuracy for speed.

Whether cuBLAS attempts emulation at all is governed by a separate strategy. The default strategy, PERFORMANT, uses emulation "whenever it provides a performance benefit", while EAGER uses it "whenever possible" [8], and for problems "too small to benefit from emulation", NVIDIA's blog adds, "cuBLAS heuristics" choose native FP64 instead [6]. From the user's side, a call can thus end on native FP64 in two ways, as cuBLAS either declines to emulate it or starts the emulation and then falls back.

### C. Opting in

Opting in requires no change to the program. Setting `CUBLAS_EMULATE_DOUBLE_PRECISION=1` enables the emulation "without any code changes", `CUBLAS_EMULATION_STRATEGY` overrides the strategy, and a third variable selects which special values the emulation must preserve [8]. According to NVIDIA, the choice "will also apply to libraries that depend on cuBLAS where applicable" [9], so programs that reach cuBLAS only through other libraries are affected as well.

Memory is where the documentation asks for more than an environment variable. The emulation needs a workspace whose size "depends on problem size, mantissa bit count, GPU architecture, and library version, but never exceeds 8 GB", and the documentation recommends reserving it with `cublasSetWorkspace`, "which can be used to guarantee reproducible results and improve performance". Without such a workspace, "the library dynamically allocates additional memory with cudaMallocFromPoolAsync() (from a library-managed CUDA memory pool scoped to the current device)", and "if this allocation fails, the library falls back to non-emulated routines for affected calls" [8]. The release notes of CUDA 13.4 Update 1, which ships the cuBLAS version we test, go further, stating that this memory now comes from "a cuBLAS-managed, per-device CUDA memory pool that retains memory across stream synchronizations", which "avoids repeated allocation overhead without requiring users to tune the default memory pool" [29]. Finally, a workspace set by the user does not persist indefinitely, because `cublasSetStream` "unconditionally resets the cuBLAS library workspace back to the default workspace pool" [8].

## IV. Problem Formulation

A single environment variable is all that cuBLAS asks of a user who wants FP64 emulation. Once `CUBLAS_EMULATE_DOUBLE_PRECISION=1` is set, every double-precision matrix multiplication that reaches cuBLAS becomes a candidate for emulation, and cuBLAS decides by itself, call by call, whether to run it on the GPU's FP64 units or on its integer tensor cores. The program, however, sees only the result. On consumer GPUs, whose native FP64 throughput is low, this hidden choice could be worth a great deal or very little, and the aim of this study is to find out which.

**Research question.** When a user enables cuBLAS's automatic FP64 emulation without changing their code, do FP64 GEMM-heavy workloads with non-square shapes and wide value ranges run faster on consumer GPUs of three architecture generations, how accurate are the results, and how often does cuBLAS fall back to native FP64?

Since the decision is made inside the library, we treat cuBLAS as a black box. Each call computes $C = AB$, with $A \in \mathbb{R}^{m\times k}$ and $B \in \mathbb{R}^{k\times n}$, and we control four of its inputs: the GPU, the shape $(m, n, k)$, the values in $A$ and $B$, and the settings the user changes. In return, we observe how long the call takes, how accurate its result is, and which path cuBLAS chose. There are three such paths. cuBLAS may emulate the product, decline to emulate it and run native FP64 from the start, or begin the emulation and then fall back to native FP64. Section V-C explains how we tell them apart.

Of these inputs, the user's settings need the most care, because the zero-code promise is not the whole story. The cuBLAS documentation also recommends giving the library a workspace for the emulation, and a second variable controls how readily it emulates at all (Section III-C). We therefore distinguish two levels of effort beyond leaving emulation off. A *zero-code* user sets the environment variable and nothing else, while a user who applies the *one-line fix* also changes how the emulation obtains its memory, either by changing a single setting of the GPU's memory pool so that it keeps freed memory or by handing cuBLAS a workspace, which also requires allocating a buffer large enough for the emulation. Either level can be combined with the EAGER strategy, which a user selects with a second environment variable and under which cuBLAS emulates wherever it can instead of only where its heuristic expects a gain; even then, the run-time fallback still applies.

Speed is judged against the alternative the user already has, native cuBLAS DGEMM on the same GPU with the same shape and inputs. With $t_{\text{native}}$ and $t_{\text{mode}}$ the times per call, measured on the same clock (Section V-C), the speedup is

$$S = \frac{t_{\text{native}}}{t_{\text{mode}}}, \tag{1}$$

so that $S > 1$ means the user gains from emulation and $S < 1$ means the switch has made the program slower. We record $S$ on two clocks: GPU time, the work done on the device as measured by CUDA events, and wall-clock time, what the application waits for when the host synchronises after each call. Normally the two agree. They part ways, however, when memory has to be mapped again before a call can run,

which the zero-code path requires after every synchronisation (Section VI-D).

A speedup is worth little if it costs accuracy, and NVIDIA states that it does not; the framework guarding the emulation chooses its precision "to maintain accuracy equal or better than FP64" [8]. We test that promise by measuring the error of each emulated product $\hat{C}$ against a reference computed in extended precision on the host, and comparing it with the error of native DGEMM on the same inputs. Both use the componentwise error

$$e = \max_{(i,j)} \frac{|\hat{c}_{ij} - c_{ij}|}{(|A|\,|B|)_{ij}}, \tag{2}$$

in which $\hat{c}_{ij}$ is an entry of the product being judged, emulated or native, $c_{ij}$ is the reference, and the maximum runs over a sample of entries. Where $(|A|\,|B|)_{ij}$ is zero, as for the rows of an injected block of zeros, we divide by one instead, so that the metric becomes the absolute error. Because each entry's error is measured against the size of the terms that form it, cancellation within a dot product does not inflate the metric. In the end-to-end workload, a randomised SVD, a user cares instead about the singular values and about how well the computed basis captures the matrix, so there we compare the singular-value error and the relative residual with those of the native run.

**Hypotheses.** Before collecting any data, we turned the question into four hypotheses. The first follows the large speedups that NVIDIA [7] and Uchino et al. [12] report for square products. The second tests a critique from AMD, whose Nick Malaya argued, as reported by HPCwire, that performance drops below native FP64 when a workload does not have square matrices [10]. The last two concern the safeguard that is meant to keep emulation accurate:

- **H1.** For large, square, well-scaled matrices, emulation beats native DGEMM by a large factor on all three GPUs, whose native FP64 throughput is low, and its accuracy at least matches native.
- **H2.** For non-square and skinny shapes typical of real codes, the speedup shrinks and can fall below native.
- **H3.** Inputs with wide exponent ranges trigger the fallback to native FP64 often enough to erase much of the end-to-end gain, and the fallback is invisible to the user except through timing.
- **H4.** Under the default precision setting, the error of the final answer in our workload stays at the level of native FP64.

**Scope.** We study three consumer GPUs, one each from the Ampere, Ada Lovelace, and Blackwell generations, and a single release of the library, cuBLAS 13.8.0.4 from CUDA 13.4 Update 1; three earlier releases appear only where we compare versions. We use cuBLAS as shipped, neither modifying it nor implementing emulation ourselves. Our objective, then, is to measure what each level of effort delivers on each GPU across shapes and value ranges, to explain when and why cuBLAS emulates, declines, or falls back, and to test the four hypotheses against those measurements.

## V. Methodology

This section describes the platform (V-A), how the levels of effort of Section IV become execution modes (V-B), how we timed and classified each call (V-C), the inputs and the accuracy reference (V-D), and the end-to-end workload (V-E).

### A. Hardware and software

All measurements come from a single server that holds one GPU of each consumer architecture we study, alongside an AMD EPYC 7C13 processor with 64 cores, 512 GiB of memory, and Ubuntu 22.04. The RTX 3090 (Ampere, compute capability 8.6) is present as two identical boards, and each experiment ran entirely on one of them. The RTX 4090 (Ada Lovelace, 8.9) has been modified to carry 48 GiB of memory instead of the stock 24 GiB. The modification changes only the memory capacity, and since no run used more than 13.1 GiB on this GPU, the extra capacity played no part in our results. The RTX 5060 Ti (Blackwell, 12.0) has 8 GiB of memory. Native DGEMM reached 0.540, 1.230, and 0.356 TFLOPS on the three GPUs in our square-GEMM runs, which sets the bar that emulation has to clear.

The software stack consists of driver version 615.71.09 and CUDA 13.4 Update 1, whose cuBLAS 13.8.0.4 we use throughout; for the comparison of versions, we loaded cuBLAS 13.2.2.2, 13.4.1.3, and 13.6.0.2, which shipped with CUDA 13.1, 13.2, and 13.3 respectively, through the library search path. cuBLAS numbers its releases separately from CUDA, so that cuBLAS 13.4.1.3, for example, predates CUDA 13.4. The workload runs in Python with CuPy 14.2.0 and NumPy 2.4.6, and we traced selected runs with Nsight Systems 2026.3.2.

We had exclusive use of the GPUs during collection, and we isolated each measurement further by pinning its process to 12 dedicated physical cores and by checking that the GPU was idle before every configuration. Without root access, however, we could not lock the clocks, so we recorded the clock and throttle state throughout instead.

### B. Execution modes

Each level of effort from Section IV becomes an execution mode, and since cuBLAS reads its emulation settings only when it initialises, every mode runs in a process of its own. A mode pairs a strategy, either cuBLAS's default or EAGER (Section III-B), with one of the memory settings in Table I. Native mode sets nothing and provides the baseline for every speedup, whereas zero code leaves the program unchanged and lets cuBLAS obtain the memory its emulation needs by itself.

The one-line fix can be applied in two ways, and we measure both. The first, the *pool-kept* variant, raises the release threshold of the device's default memory pool to its maximum with `cudaMemPoolSetAttribute`, after obtaining the pool with `cudaDeviceGetDefaultMemPool`, so that memory freed after a call stays mapped for the next. The second, the *workspace* variant, passes cuBLAS a buffer through `cublasSetWorkspace`, sized at the workspace

cuBLAS reports for that shape plus a margin of 32 MiB. Strictly, neither variant is a single line, since the first must also obtain the pool and the second must allocate the buffer and know how large to make it, but each changes a single setting, and we keep one name for both because they remove the same cost. For the special-value experiments alone, a further EAGER variant clears the special-value support mask, which releases cuBLAS from preserving infinities and NaNs.

### C. Timing, path detection, and quality control

All GEMM measurements come from a C++/CUDA benchmark that times one configuration per process. Its inputs are generated deterministically from a seed, so that outputs can be compared bit for bit across processes and GPUs. After a warm-up of at least one second, the benchmark times calls for about three seconds and reports the median GPU time of a call, from CUDA events. It then measures the wall-clock time per call twice, as the mean over a series of calls that each end with a host synchronisation and over the same number of calls run back to back with one synchronisation at the end. Across the repeats of a configuration, we report the median. Every experiment ran three times, with the order of configurations rotated between repeats. The only exceptions, which ran once, are the comparison of versions, the fine grid around the fallback threshold together with the kernel timings of the precision settings, and a decision map that records the path and the GPU time of each call from only three timed calls after a shorter warm-up, without an accuracy reference or wall-clock timing.

Which clock we report depends on the mode. With the one-line fix, the two clocks agree, as across 572 comparisons the speedups they give differ by at most 3.4 %, so for these modes we report GPU time. With zero code they diverge, and we report the wall-clock time with a synchronisation after every call, because that is what an application that synchronises after each product experiences, and Section VI-D compares it with calls run back to back. Zero-code wall times also vary between runs of the same configuration, by 8 % in the median case and by more than 20 % in 58 of 352 configurations, and we therefore pool all runs of a zero-code configuration and report their median.

Timing alone cannot tell an emulated call from one that fell back, so the benchmark also records, through the CUDA Profiling Tools Interface (CUPTI), the kernels each call launches. A call counts as emulated when more than half of its kernel time falls in cuBLAS's emulation kernels, and as fallen back when it launched further kernels but spent most of its time in a native FP64 GEMM kernel; otherwise, cuBLAS declined to emulate it. The kernel names also show which scheme ran. In all 1612 full kernel records that we kept of emulated calls in cuBLAS 13.8.0.4, the call launched kernels named for Ozaki-II and its moduli, such as `oz2_int8_dgemm`, which appear in none of the 465 records from the three earlier releases. The benchmark also reads the number of mantissa bits each call used from a counter that a program can register with `cublasSetFixedPointEmulation MantissaBitCountPointer` [8]. We set it to a sentinel value before the last timed call, so that a call which leaves it untouched can be recognised. To explain the mechanisms behind the results of Section VI-D, we further traced selected configurations with Nsight Systems, which records every CUDA API call, and timed the allocation pattern of the memory pool in isolation.

Because we could neither lock the clocks nor have the host to ourselves, an automated check examined every run for missing rows, foreign processes on the GPU, changes of path or output between repeats, non-determinism, throttling, and excessive spread. The data passed it well. All 8285 benchmark rows completed, none of the rows showed hardware slowdown, hardware thermal, or power-brake throttling, and only 8 carry the milder software thermal flag. No foreign process ran on a GPU before any configuration, and every output was reproduced bit for bit when its call was repeated. We reran the 31 configurations that the check flagged, with three further repeats each, and outside zero code the results were consistent within about 2 %.

### D. Inputs and accuracy

Uniform inputs, with entries drawn from $[-1, 1)$, serve as the well-scaled case throughout, and three families widen the exponent range under a parameter $\varphi$. The first, which we call *ozaki*, uses the generator that is standard in the Ozaki-scheme literature [2], [3], $a_{ij} = (u_{ij} - 0.5)\exp(\varphi\, g_{ij})$, with $u_{ij}$ uniform on $[0, 1)$ and $g_{ij}$ standard normal. The other two multiply uniform matrices by factors $10^{\varphi t}$, with $t$ uniform on $[-1, 1]$, but along different directions. Scaling the rows of $A$ and the columns of $B$, which we call *rowcol*, gives all terms of one dot product the same scale, so the range within a dot product stays narrow. Scaling the columns of $A$ and the rows of $B$ instead, which we call *inner*, acts along the summation index, and the terms of a single dot product can then span up to $4\varphi$ orders of magnitude. For the special-value experiments, we also inject NaN, $\pm\infty$, subnormal, very large, or overflowing values, or a block of zeros, into otherwise uniform inputs.

An accuracy reference must be more accurate than anything it judges, so the benchmark recomputes 4096 sampled entries of $C$ in 80-bit extended precision on the host. We checked this reference against a second evaluation with the compensated dot product Dot2 of Ogita, Rump, and Oishi [30]. In the error metric of Section IV, the two agree to within $2.3 \times 10^{-19}$ on uniform inputs and $5.2 \times 10^{-18}$ on the wide-range families, and they differ more only for injected subnormals, whose products underflow in Dot2's double-precision arithmetic. For the native comparison, each process also computes the same product on a second cuBLAS handle with the `CUBLAS_COMPUTE_64F_PEDANTIC` compute type, which the documentation describes as using "64-bit double precision floating point arithmetic for all phases of calculations" [8]. This product supplies the native error on identical inputs and the reference for bitwise and special-value comparisons.

TABLE I
EXECUTION MODES. EACH OF THE THREE EMULATION SETTINGS RUNS UNDER THE DEFAULT STRATEGY AND UNDER EAGER, WHICH ADDITIONALLY SETS `CUBLAS_EMULATION_STRATEGY=EAGER`; WITH NATIVE, THIS GIVES SEVEN MODES.

| memory setting | what the user changes | emulation workspace |
|---|---|---|
| native (baseline) | nothing | none |
| zero code | sets `CUBLAS_EMULATE_DOUBLE_PRECISION=1` | obtained by cuBLAS |
| one-line fix, pool kept | as zero code, plus a raised release threshold for the pool (`cudaMemPoolSetAttribute`) | obtained by cuBLAS; freed memory stays in the pool |
| one-line fix, workspace set | as zero code, plus a buffer passed with `cublasSetWorkspace` | the user's buffer |

### E. Workload: randomised SVD

A GEMM benchmark cannot show whether the gains survive inside an application, so we also run a randomised SVD, a natural candidate because its work is dominated by large matrix products. Following Halko et al. [31], it combines randomised subspace iteration (their Algorithm 4.4), with $q$ power iterations and a QR re-orthonormalisation after each product, with a direct SVD of the small matrix $B = Q^{\mathsf{T}}A$ (their Algorithm 5.1). Its GEMMs thereby cover several of the shapes we sweep in isolation, as the first sketch $A\Omega$ is skinny, the products with $A^{\mathsf{T}}$ are transposed, and $Q^{\mathsf{T}}A$ is short and wide. We implemented it in CuPy, so every large GEMM goes through cuBLAS and every QR factorisation through cuSOLVER.

The test matrices are dense, $16384 \times 8192$, and built as $A = U\,\mathrm{diag}(\sigma)V^{\mathsf{T}}$ from orthonormal factors $U$ and $V$ obtained by QR factorisation of Gaussian matrices. Two spectra are adapted from the test matrices of Tropp et al. [32]. Both keep ten unit singular values, followed either by exponential decay, $\sigma_j = 10^{-0.25(j-10)}$ (*exp*), or by polynomial decay, $\sigma_j = (j-9)^{-2}$ (*poly*). A third matrix, *lrnoise*, has rank 50 plus Gaussian noise whose entries have standard deviation $10^{-10}/\sqrt{16384}$. Finally, two badly scaled variants, *scaled4* and *scaled8*, multiply the columns of *exp* by $10^{\varphi t_j}$ with $\varphi = 4$ and $8$, which places a wide exponent range along the inner dimension of the first sketch. On each matrix we vary the sketch size $\ell \in \{64, 128, 256, 512, 1024\}$ and the number of power iterations $q \in \{0, 2\}$.

Besides native FP64, the workload runs under zero code and under the pool-kept fix, each with the default strategy and with EAGER, and, as lower-precision comparisons, in FP32 and in TensorFloat-32 (TF32). The fix here is the pool-kept variant, which, unlike a workspace, also takes effect under CuPy (Section VI-D). Each process handles one matrix in one mode; every $(\ell, q)$ pair runs twice to warm up and five times timed, with a device synchronisation at the end of each run, and we report the median wall-clock time, taken over three repeats with the order of modes rotated. Accuracy is computed on the host, in double precision, from the last timed run. We report the largest relative error among the ten largest singular values, measured against LAPACK's `dgesdd` on the stored matrix, and the relative residual $\|A - QB\|_F / \|A\|_F$, which shows how well the computed basis captures $A$.

### F. Reproducibility

The benchmark, the workload, the analysis scripts, and the raw results are available at https://github.com/Taiz2000/fp64-emulation-data. Each run of the GEMM benchmark records the hash of the benchmark binary, the GPU and core pinning, and a once-per-second log of the GPU's clocks, power, and memory use, and the analysis scripts regenerate the tables and figures from these results.

## VI. EXPERIMENTS

We present the results in the order in which cuBLAS makes its decisions: whether it emulates large products (VI-A), how its default strategy treats other shapes (VI-B), and when it falls back at run time (VI-C). We then turn to what emulation costs a user who changes no code (VI-D), to the end-to-end workload (VI-E), and to a summary of what each level of effort delivers (VI-F). Unless we state otherwise, speedups are measured in GPU time with the one-line fix and in wall-clock time with zero code, as explained in Section V-C.

### A. Large square GEMMs

We begin with the case most favourable to emulation, square products of uniform matrices with $n \in \{512, 1024, 1536, 2048, 3072, 4096, 6144, 8192, 12288, 16384\}$ (up to 12288 on the RTX 5060 Ti), run in all seven modes. Fig. 1 shows each GPU under the strategy that emulates there, which is EAGER on the RTX 3090, where the default strategy never emulates (Section VI-B), and the default strategy on the other two.

**Emulation wins at every size it emulates, and its lead grows with $n$.** At $n = 4096$, it ran $11.1\times$, $10.5\times$, and $13.2\times$ faster than native DGEMM on the RTX 3090, 4090, and 5060 Ti, and its speedup peaked at $14.5\times$ ($n = 12288$), $15.1\times$ ($n = 12288$ and $16384$), and $18.8\times$ ($n = 8192$) respectively. Since native DGEMM runs at a constant rate on these GPUs for $n \geq 4096$, the growth comes from the emulation itself. It also fits the structure of the schemes in Section III-A, because converting the operands touches each of the $O(n^2)$ entries, whereas the integer GEMMs perform $O(n^3)$ work, so the conversion weighs less as $n$ grows.

**Emulation is also more accurate, and identical across GPUs.** Its componentwise error ranged from $4.4 \times 10^{-18}$ to $2.4 \times 10^{-17}$ across all sizes and GPUs, against $2.4 \times 10^{-16}$ to $3.4 \times 10^{-16}$ for native DGEMM, which makes it at least ten times lower in every case and still well above the $2.3 \times 10^{-19}$ resolution of our reference. ADP chose 58 to 60 mantissa bits

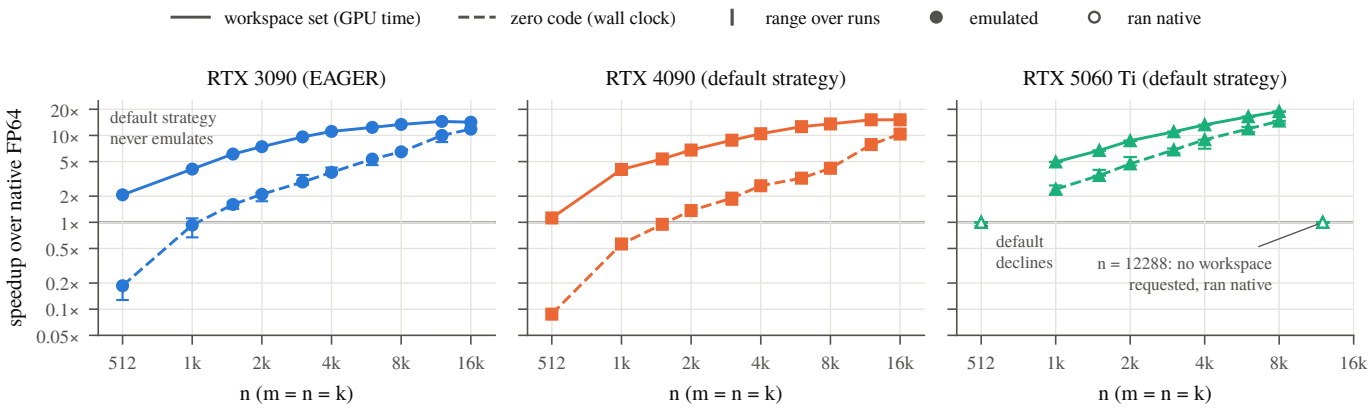



Fig. 1. Speedup over native DGEMM for square GEMMs with uniform inputs. Solid lines: one-line fix (workspace set), GPU time. Dashed lines: zero code, wall-clock time with a synchronisation after every call. Bars: range over the runs. Filled markers: emulated; hollow: ran native. The RTX 3090 panel uses EAGER, because the default strategy never emulates on that GPU; the other panels use the default strategy.

for these inputs. Since the integer GEMMs are exact, rounding enters only when the inputs are converted and the result is reconstructed, rather than at each of the $n$ multiply-adds of a native dot product. Moreover, the emulated results were identical bit for bit across the three GPUs. All 251 emulated configurations that ran on more than one GPU, in this and the later experiments, produced a single output hash. Finally, transposing either operand changed the speedup by at most 3.3 %.

**The RTX 5060 Ti shows the first silent exception.** Its default strategy declined $n = 512$, where EAGER ran 2.5× faster than native, and at $n = 12288$ every mode, EAGER included, ran native. For that size cuBLAS requested no emulation workspace at all, although it had requested 4928 MiB at $n = 8192$. At $n = 12288$ the other two GPUs requested 6.3 GiB, which together with the 3.4 GiB that the three matrices occupy exceeds the card's 8 GiB. The call nevertheless succeeded and returned a native result, without any error or warning that emulation had been skipped.

**Zero code keeps only part of the gain.** The dashed curves in Fig. 1 lie well below the solid ones. On the wall clock, with a synchronisation after every call, zero code retained between 9 % and 83 % of the speedup of the one-line fix on the RTX 3090 under EAGER, between 8 % and 69 % on the RTX 4090, and between 49 % and 78 % on the RTX 5060 Ti, and on every GPU the share rose steadily with $n$. At small sizes zero code even made the program slower than native, at 0.19× for $n = 512$ on the 3090, where it also needed EAGER, and at 0.09× and 0.56× for $n = 512$ and 1024 on the 4090. At the next size up, $n = 1024$ on the 3090 and $n = 1536$ on the 4090, zero code reached 0.93× and 0.95×, but the three runs ranged from 0.67× to 1.11× and from 0.87× to 1.03×, so these lie within the session-to-session variation of zero code. Both variants of the one-line fix removed the loss, and they agreed with each other to within 1.6 %. On large, well-scaled products, emulation thus delivers what H1 predicts on all three GPUs, but whether a user receives it depends on the strategy and on the memory setting, which the rest of this section examines.

### B. Shapes and the default strategy's decisions

To test H2, we varied one dimension while the other two stayed at 8192: the width $n$ of skinny products (8 to 2048), the inner dimension $k$ of panel updates (16 to 2048), and the height $m$ of short-wide products (8 to 512). The same grid contains Gram-type products with $m = n \in \{32, 128, 512\}$ and $k$ up to 131072, three odd sizes, two squares, and the six GEMM shapes of our randomised SVD, 49 shapes in all. A separate decision map records which path the default strategy and EAGER take, and the GPU time of each call, on the cube $m, n, k \in \{16, 64, 256, 1024, 4096, 16384\}$ and on slices through it in steps of $\sqrt{2}$, 290 shapes in total (287 on the 5060 Ti). Where the default declines a shape, its own run gives the native time against which we compare EAGER.

**Below a varying dimension of 48 to 192, emulation loses heavily.** Under EAGER, with the other two dimensions at 8192, emulation beat native once the varying dimension reached 96 to 192 for skinny products, 48 to 64 for panels, and 128 for short-wide products (Fig. 2). Below these points the losses were large. At $n = 8$, a skinny product ran at 0.06× to 0.41× of native speed, a panel with $k = 16$ at 0.31× to 0.51×, and a Gram product with $m = n = 32$ and $k = 8192$ at 0.05× to 0.17×.

**The cause is a cost that does not shrink with the small dimension.** On the RTX 3090, for example, an emulated skinny product took 24.9 to 25.1 ms for every $n$ from 8 to 128, while native DGEMM took from 1.9 to 28.2 ms over the same range. This is consistent with a fixed cost of converting the matrices that stay large, as in every family one operand (skinny and short-wide) or the output (panel) remains $8192 \times 8192$ whatever the small dimension is. Because the speedup divides by native time, it also inherits the steps in cuBLAS's choice of native kernel. On the 3090, native DGEMM took 10.2 ms at $n = 16$ but only 7.0 ms at $n = 32$, which produces the bump in Fig. 2.

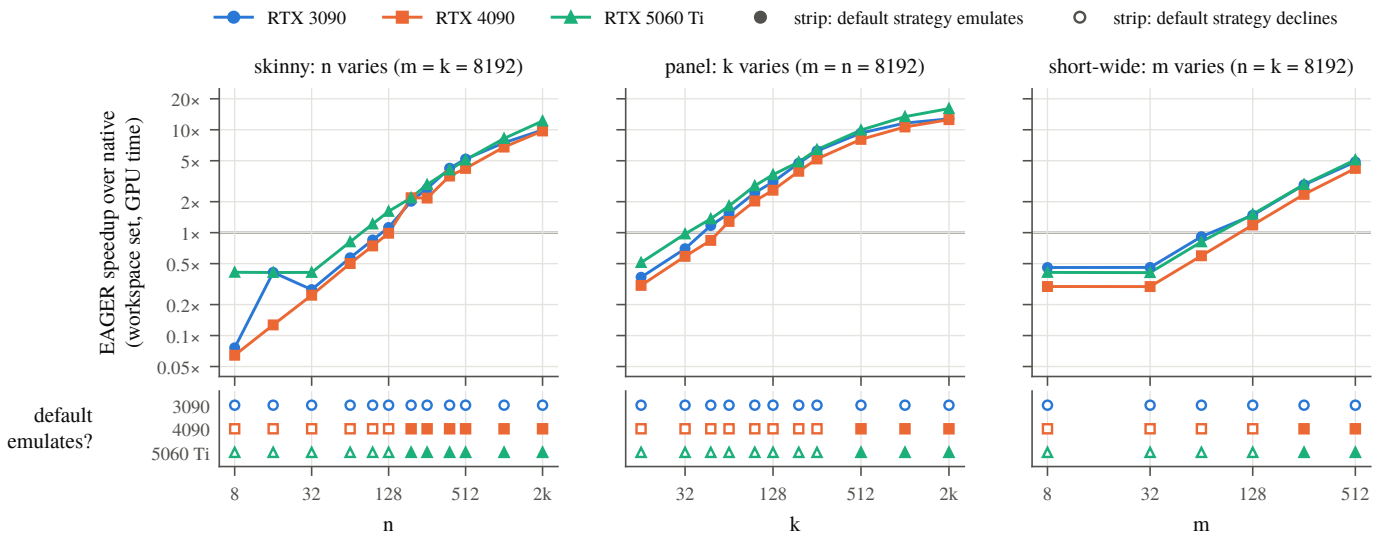



Fig. 2. Speedup of EAGER over native (one-line fix, GPU time) as one dimension varies and the other two stay at 8192. The strip under each panel is filled where the default strategy emulates that shape. Speedups divide by cuBLAS's native time, which changes with its kernel choice; the RTX 3090 bump at $n = 16$ comes from a slower native kernel at that size, and the RTX 5060 Ti's native time is constant from $n = 8$ to 32.

**The default strategy never emulates on the RTX 3090.** None of its 1340 measurements on this GPU, across all experiments and all four cuBLAS releases, took the emulated path, even though the documentation lists the 3090's compute capability as supported. In the decision map it declined all 290 shapes, although EAGER was faster at 123 of them and more than 1.5× faster at 109. On this GPU, then, a user must select EAGER to obtain any emulation at all.

**On the other two GPUs the default is mostly right, but conservative.** It emulated 90 of 290 shapes on the RTX 4090 and 74 of 287 on the 5060 Ti, and where it declined, EAGER was slower at 175 of the 200 declined shapes on the 4090 and at 157 of 213 on the 5060 Ti; only 15 and 29 of the declined shapes would have run more than 1.5× faster. Its threshold, however, sits above the break-even point. On both GPUs it started to emulate at $n = 192$, $k = 512$, and $m = 256$, where EAGER already wins from $n = 96$ to 192, $k = 48$ to 64, and $m = 128$. On the 5060 Ti, a further 15 shapes, each with two dimensions of 16384 or the cube $11585^3$, ran native even under EAGER, in the same way as $n = 12288$ in Section VI-A.

**In exchange, the default chose no losing shape where we measured native.** None of the 142 configurations that it emulated and for which we also measured native DGEMM ran below native in GPU time. The decision map has no native run of the shapes that the default emulates, so it is not part of this count. AMD's critique therefore holds for the algorithm, which in these families loses below a varying dimension of 48 to 192, but the default strategy keeps a user away from those shapes, at the price of some declined wins. This finding covers GPU time only, and Section VI-D shows that with zero code several of the shapes it emulates still run below native on the 4090.

### C. Run-time fallback and the precision cliff

To test H3, we widened the exponent range at $4096^3$ with the three families of Section V-D, letting $\varphi$ run from 0 to 8 for *ozaki* inputs, from 1 to 12 for *inner*, and from 2 to 16 for *rowcol*. For *inner* we also varied $k$ from 256 to 16384, and a finer grid of 56 configurations ($\varphi$ from 7.25 to 8, $k$ from 256 to 16384) covers the region where fallback begins. We also ran the precision settings that a user can choose instead of the default, fixed mantissa counts from 24 to 96 bits and dynamic caps of 64 and 79 bits, on uniform inputs and on *inner* inputs with $\varphi = 6$.

**The bits needed follow the range inside the dot products.** For *ozaki* inputs, the mantissa bits ADP required rose from 58 at $\varphi = 0$ to 102 at $\varphi = 4$, and for *inner* inputs at $k = 4096$ from 64 at $\varphi = 1$ to 106 at $\varphi = 7.75$. Row and column scaling, in contrast, left them at 60 bits even at $\varphi = 16$, as the shared scaling factors of Section III-B predict. For *inner* inputs the bits also fell slightly as $k$ grew (Fig. 3a), at $\varphi = 8$ from 111 at $k = 256$ to 107 at $k = 16384$. This follows from how ADP counts them, as 53 bits plus the exponent span capacity of Schwarz et al., which adds the exponents of the largest entries in a row of $A$ and in a column of $B$ and subtracts that of the largest term of their dot product [7]. Because the *inner* inputs scale the columns of $A$ and the rows of $B$ independently, a longer dot product is more likely to contain a term in which both factors are large, so its largest term comes closer to the product of the two largest entries and fewer bits are needed. Up to the limit, emulation stayed fast. Between 58 and 106 bits its speedup fell by about a quarter, from 11.1× to 8.0× on the 3090, from 10.5× to 7.8× on the 4090, and from 13.2× to 10.1× on the 5060 Ti (Fig. 3b), while its error at $\varphi = 7.75$, $1.8 \times 10^{-16}$, remained well below native's $3.9 \times 10^{-15}$.

**Beyond about 107 bits, cuBLAS falls back, identically**

TABLE II

PRECISION SETTINGS, UNIFORM INPUTS, $4096^3$, EAGER WITH THE WORKSPACE SET, ALL MEASURED IN ONE RUN. SPEED RELATIVE TO THE DEFAULT DYNAMIC SETTING, WHICH ITSELF RAN 11.1×, 10.5×, AND 13.2× FASTER THAN NATIVE DGEMM (SECTION VI-A), AND COMPONENTWISE ERROR (IDENTICAL ON THE THREE GPUS); NATIVE ERROR IS $2.4 \times 10^{-16}$ TO $3.4 \times 10^{-16}$.

| setting | mantissa bits | error | RTX 3090 | RTX 4090 | RTX 5060 Ti |
|---|---|---|---|---|---|
| fixed | 24 | $3.1 \times 10^{-9}$ | 2.49× | 2.30× | 2.45× |
| fixed | 48 | $2.1 \times 10^{-16}$ | 1.65× | 1.64× | 1.76× |
| fixed | 53 | $8.2 \times 10^{-18}$ | 1.49× | 1.50× | 1.61× |
| dynamic, cap 64 | 59 used | $8.2 \times 10^{-18}$ | 1.18× | 1.18× | 1.26× |
| dynamic, default | 59 used | $8.2 \times 10^{-18}$ | 1.00× | 1.00× | 1.00× |
| fixed | 96 | $1.4 \times 10^{-17}$ | 0.75× | 0.78× | 0.79× |

**on all three GPUs.** Every configuration that needed 106 bits or fewer was emulated, every one that needed 108 or more fell back, and those that needed exactly 107 went either way (Fig. 3a). Across 83 configurations, the required bits and the decision never differed between the GPUs, so although the documentation allows the default maximum to vary with the architecture (Section III-B), it did not vary across our three. At 107 bits the outcome was not even monotone in the range. At $k = 2048$, $\varphi = 7.7$ and 7.75 fell back while $\varphi = 7.8$ was emulated, all three reporting 107 bits, so the decision cannot depend on the reported bit count alone. A fallback itself costs little. The call ran at 0.98× to 1.00× of native speed, and its error stayed within 0.94 to 1.00 times that of native DGEMM.

**Special values fall back too, but overflow is handled better than by native DGEMM.** Inputs containing NaN or $\pm\infty$ always fell back (0.97× to 1.04× of native), as in the ADP workflow of Section III-B, and they did so even with the special-value mask cleared. Subnormal, very large, and overflowing inputs and zero blocks, by contrast, were emulated at full speed, 6.7× to 8.7× at $2048^3$. For overflow, emulation was more faithful than native. In a test with seven inputs, we scaled one row of $A$ and one column of $B$ by $10^{200}$, so that the exact value of one output entry, computed with rational arithmetic from the scaled inputs, lay between $3 \times 10^{400}$ and $3 \times 10^{401}$ in magnitude. Emulation under EAGER returned an infinity of the correct sign for all seven, native GPU DGEMM for one, returning an infinity of the wrong sign for the other six, and OpenBLAS on the CPU returned NaN for all seven. The first input ran on all three GPUs, with identical results, and the other six on the RTX 4090. In native arithmetic the individual products already overflow, some to $+\infty$ and some to $-\infty$, so the result no longer carries the sign of the exact sum, and OpenBLAS returns their sum, NaN. The emulation instead works on scaled integers and overflows only once, when it applies the scaling factors to the result.

**The default precision setting costs about a third of the speed.** With uniform inputs at $4096^3$, the default dynamic mode used 59 bits, whereas a fixed mantissa of 53 bits gave the same error, $8.2 \times 10^{-18}$, at 1.49×, 1.50×, and 1.61× the speed of the default (Table II). Kernel timings show where the difference goes. The INT8 GEMMs took the same time under every dynamic cap, but the modular reduction that precedes them is sized by the cap rather than by the bits a call needs, and under the default it took 35 % to 43 % of each call.

**Fixed bits, however, can lose accuracy silently.** On the *inner* input with $\varphi = 6$, which needs 94 bits, 53 fixed bits still beat native accuracy ($8.3 \times 10^{-16}$ against $3.3 \times 10^{-15}$), but 48 bits gave $2.4 \times 10^{-14}$, seven times worse than native, with no fallback and no warning. A dynamic cap below the required bits behaved differently, as with caps of 64 or 79 bits the same input fell back to native instead of losing accuracy. Dynamic precision, whether by default or under a user cap, never lost accuracy in our GEMM experiments. In all 2624 measurements in which cuBLAS emulated a product in this way and we had a reference, across the three GPUs and four releases, the error was at most 0.53 times that of native DGEMM.

**Older releases fall back earlier.** The three releases from CUDA 13.1 to 13.3 predate Ozaki-II in cuBLAS [29], and they already fell back on the *inner* inputs with $\varphi = 7.5$ and 7.75 at $4096^3$, at 104 and 106 bits, which cuBLAS 13.8.0.4 emulates. On uniform inputs at $n = 4096$ they were also slower, reaching 8.5×, 8.7×, and 9.4× under EAGER, 77 %, 84 %, and 71 % of the speed that cuBLAS 13.8.0.4 reached in the same run.

These results confirm the second half of H3 for a user who changes no code. A call that declines and a call that falls back both return success and a result of native accuracy, and cuBLAS raises no error or warning. The one sign inside the library is the counter of Section V-C, whose contents the documentation does not specify for calls that do not emulate. Across our GEMM experiments, it stayed unwritten in all 2722 measurements that cuBLAS declined and was written in all 4447 that entered emulation. After a fallback, it held $-1$ for inputs with NaN or infinity and otherwise the bits the call would have needed. Under the default limit of cuBLAS 13.8.0.4, emulated calls reported at most 107 bits and fallbacks at least 107, so the counter identifies every decline, and every fallback except those at exactly 107 bits. A program must request it, however, and without a code change only the timing, or a profiler's record of the kernels launched, reveals what happened. The first half, a fallback frequent enough to erase the gain, depends on the inputs. In our GEMM grids it took a range inside the dot products that needed at least 107 mantissa bits, far beyond what row or column scaling produces, or a NaN or infinity among the inputs, and Section VI-E tests how often this happens in a workload.

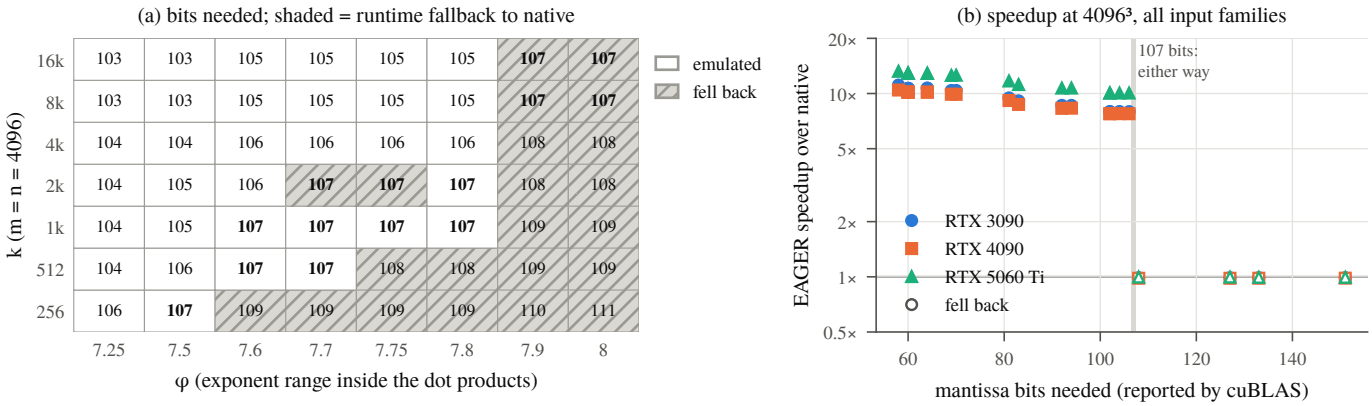



Fig. 3. (a) Mantissa bits that ADP reports for *inner* inputs at $m = n = 4096$, by inner dimension $k$ and range parameter $\varphi$; shaded, hatched cells fell back to native, and bold numbers mark 107 bits, where both outcomes occur. The grid is the same on all three GPUs. (b) Speedup of EAGER over native at $4096^3$ against the bits needed, for all input families; hollow markers fell back.

### *D. The zero-code memory penalty*

This subsection explains why zero code keeps only part of the gain, and in doing so finds that cuBLAS 13.8.0.4 does not manage the emulation's memory in the way its release notes describe.

**Every emulated call allocates its workspace afresh.** A trace of the RTX 4090 at $4096^3$ under the default strategy contains one `cudaMallocAsync` and one `cudaFreeAsync` per GEMM call, and neither once a workspace is set. The allocations took 6.1 ms per call on average, and the synchronisation after each call took about 29 ms longer than with the workspace set, together about 35 ms per call under the profiler. Both costs come from the device's default memory pool, which, with its release threshold at zero, returns freed memory at every synchronisation, so that the next allocation has to map it again. Our microbenchmark puts this remapping at 42.7, 25.6, and 10.6 ms per GiB on the 3090, 4090, and 5060 Ti, against at most 0.005 ms per allocation, of any size, once the threshold is raised. The penalty is therefore mostly a host-side cost, and only 6 % to 39 % of it appears in GPU time, which is why the two clocks diverge for zero code.

**The penalty is paid at each synchronisation, not at each call.** Because the pool gives memory back only when the host synchronises, calls that run back to back reuse the workspace that the previous call freed. When the benchmark ran the same calls back to back with one final synchronisation, zero code reached 97 % to 100 % of the speed of the one-line fix up to $n = 4096$ on all three GPUs, and 89 % to 98 % at larger sizes. On the RTX 4090, for example, it ran 4.06× and 10.18× faster than native at $n = 1024$ and 4096, against 0.56× and 2.64× with a synchronisation after every call. Whether a program pays the penalty therefore depends on how often it synchronises, and our randomised SVD, which synchronises once at the end of each decomposition, paid it about once per decomposition, however many of its products were emulated (Section VI-E).

**The workspace size and the remap rate predict the penalty.** Multiplying the workspace that cuBLAS requests by the GPU's remap rate predicted the measured penalty to within a factor of 0.79 to 1.60, across all sizes and GPUs (Fig. 4). At $n = 4096$ on the 4090, for example, 1232 MiB at 25.6 ms per GiB predicts 30.8 ms per call, and we measured 31.7 ms. Wherever the GPUs emulated the same size, cuBLAS requested the same workspace on all three, so the differences between them come from the remap rate, which is also why the 5060 Ti loses least. The share that zero code keeps rises with $n$ (Section VI-A) for a similar reason, as the workspace grows more slowly than the arithmetic and, on the RTX 3090 and 4090, levels off at 6.3 to 6.6 GiB from $n = 12288$, while the work of the GEMM keeps growing. The same workspace bears on Malaya's remark that emulation uses "about twice the memory capacity" of native FP64 [11]. Counting the three matrices for native DGEMM, a square product together with the workspace that cuBLAS requests needed 4.2 times the memory of native DGEMM at every size from $n = 1536$ to 8192, more at smaller sizes, and 2.9 and 2.1 times at $n = 12288$ and 16384, where the workspace levels off, so his estimate holds only for the largest products.

**What we observed differs from the documentation of this version.** The release notes of CUDA 13.4 Update 1 state that the emulation takes this memory from a cuBLAS-managed pool that retains it across synchronisations (Section III-C), which would make the penalty disappear. To test the statement directly, we reset the high-water marks of the device's memory pools before each call at $4096^3$ and read them afterwards. With zero code, every emulated call took 1232 MiB from the device's default pool on all three GPUs, and when we installed another pool as the device's current pool, the allocation moved there and the default pool stayed empty. Traces of both the runtime and the driver API show one `cudaMallocAsync` per call and no call that creates a pool or allocates from a particular one. In cuBLAS 13.8.0.4, then, the workspace comes from whichever pool is current, through a plain `cudaMallocAsync`. This also explains why the pool-kept fix works, since every call still allocates 1232 MiB, but the memory now stays mapped between calls.

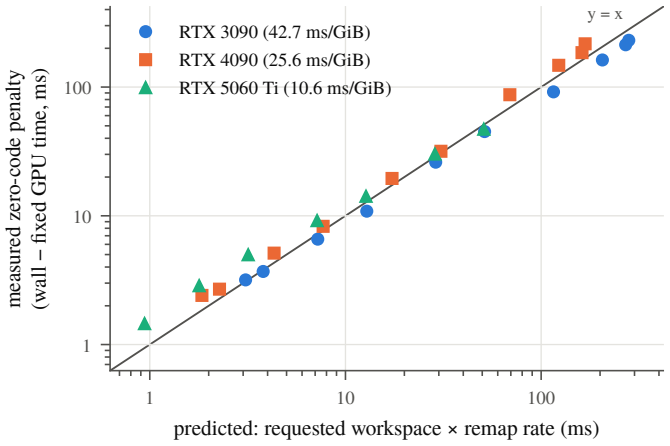



Fig. 4. Zero-code penalty per call (zero-code wall-clock time minus GPU time with the workspace set) against the prediction from the requested workspace and each GPU's remap rate, for square GEMMs from $n = 512$ to 16384. The line marks equality.

**Through CuPy, only the pool-kept fix takes effect.** A workspace set with `cublasSetWorkspace` on CuPy's cuBLAS handle was honoured when we called cuBLAS directly, and the calls then took nothing from the pool. When the same product went through CuPy, however, whether through its matrix operator or its own DGEMM wrapper, every call again took 1232 MiB. This is consistent with CuPy setting the stream on its handle before each call, because `cublasSetStream` resets the workspace (Section III-C). A CuPy user who wants the one-line fix must therefore apply it to the memory pool.

**The penalty also reaches shapes that the default strategy treats as safe.** On the RTX 4090, the default emulated a skinny product with $n = 256$ at 2.17× in GPU time, yet with zero code the same product ran at 0.48× of native on the wall clock. Thirteen shapes that the default chose to emulate ran below native in this way, among them the three GEMM shapes of our randomised SVD with $\ell = 256$, at 0.49× to 0.52×. The finding of Section VI-B thus holds in GPU time but not on the wall clock, which suggests that the heuristic does not account for the allocation.

### *E. An application: randomised SVD*

The randomised SVD of Section V-E combines the shapes of VI-B with structured test matrices and, in the *scaled* matrices, a range that stresses the precision cliff of VI-C. Table III gives its speedups for three representative settings of $\ell$ and $q$.

**Emulation pays once the sketch is wide enough.** With the one-line fix and EAGER, the decomposition of *exp* ran 0.69×, 0.59×, and 0.86× as fast as native at $\ell = 64$ and $q = 0$ on the RTX 3090, 4090, and 5060 Ti, but 2.68×, 2.25×, and 3.04× at $\ell = 512$ and $q = 2$. The losses at $\ell = 64$ match Section VI-B, since the large products of the decomposition are then skinny or short-wide with a small dimension of 64, below the break-even points of 96 to 192 and 128 for those families. The gains, in turn, are far smaller than for a single large GEMM, because only part of the work is emulated. Under EAGER, the emulation kernels took 37 % to 39 % of the GPU time of a profiled pass over all settings of *exp*, and the remainder, which runs in native FP64, limits the overall speedup.

**Zero code can make the application slower than native.** On the RTX 4090, zero code ran the decomposition at 0.62× to 0.66× of native at $\ell = 256$ and $q = 0$ for every matrix, and the one-line fix turned the same setting on *exp* into a speedup of 1.65×. The default strategy emulates the $\ell = 256$ GEMMs, and the synchronisation that ends each decomposition gives their workspace back, so the next decomposition has to map it again. On *exp*, the extra time per decomposition was 146 ms at $q = 0$ and 143 ms at $q = 2$, although the number of emulated calls per decomposition rose from two to six, so the penalty was paid about once per decomposition rather than once per call. It amounts to one or two remaps of the 3.2 to 3.3 GiB workspace that these GEMMs request, at about 85 ms each on this GPU, and exceeds the 96 ms that the whole decomposition takes with the fix. On the RTX 3090, zero code changed nothing, as the default never emulates there, while on the 5060 Ti, with its low remap rate, zero code came close to the fix (2.94× against 3.04× at $\ell = 512$, $q = 2$).

**The most badly scaled matrix loses more than half of the gain, and it is the one on which the fallback sets in.** On *scaled8*, the fix with EAGER reached 1.57× to 1.74×, 1.36× to 1.59×, and 1.39× to 1.81× on the three GPUs for $q = 2$ and $\ell$ from 256 to 1024, against 2.20× to 2.68×, 1.80× to 2.25×, and 1.93× to 3.04× on *exp*, whereas *scaled4*, whose range stays below the limit, ran within 4 % of *exp*. Measured as the gain over native, $S - 1$, every mode that emulated lost at least half of it, 51 % to 61 % wherever the decomposition stayed faster than native, and all of it for zero code on the RTX 4090 at $\ell = 256$ and $q = 2$, with or without EAGER, which fell from 1.13× and 1.12× on *exp* to 0.94× and 0.93× on *scaled8*, where the fallback adds to the memory penalty. A profiled pass over all settings shows the fallback at work. Of the 50 calls that entered emulation under EAGER (44 on the 5060 Ti), 10 fell back on *scaled8* and took 27 % to 34 % of the GPU time, whereas none fell back on *exp*. The timings identify the product that fell back. Without power iterations ($q = 0$), *scaled8* ran no more than 1.4 % slower than *exp* in every mode that emulated, on all three GPUs, so none of the products that run there, $A\Omega$, $Q^{\mathsf{T}}A$, and $QU_b$, fell back. The power iterations add $A^{\mathsf{T}}Q$ and $AQ_z$, where $Q_z$ is the orthonormal factor from the QR factorisation of $A^{\mathsf{T}}Q$, two of each per decomposition with $q = 2$, which matches the 10 fallbacks. In $A^{\mathsf{T}}Q$, each dot product runs down a single column of $A$, whose scale the shared scaling factor absorbs (Section III-B), which leaves $AQ_z$. The rows of $A^{\mathsf{T}}Q$, and hence of $Q_z$, carry the column scaling of $A$, so each term of a dot product in $AQ_z$ carries that scaling twice, and the terms span up to 32 orders of magnitude, as for the *inner* inputs at $\varphi = 8$, which fell back at the same inner dimension of 8192 (Fig. 3a). Since *scaled8* lost at most 1.4 % at $q = 0$, where no product fell back, the loss on *scaled8* came with the fallback

TABLE III

RANDOMISED SVD OF *exp* ($16384 \times 8192$): SPEEDUP OVER NATIVE FP64, WALL-CLOCK TIME. MEDIANS OF THREE REPEATS; IN THE CELLS SHOWN, THE REPEATS AGREED WITHIN 2 % FOR FP64 MODES, EXCEPT ZERO CODE WITH EAGER (UP TO 16 %), AND WITHIN 7 % FOR FP32.

| GPU | mode | $\ell = 64$, $q = 0$ | $\ell = 256$, $q = 0$ | $\ell = 512$, $q = 2$ |
|---|---|---|---|---|
| RTX 3090 | zero code | 1.00 | 1.00 | 1.00 |
| | one-line fix | 1.00 | 1.00 | 1.00 |
| | zero code, EAGER | 0.24 | 0.90 | 2.31 |
| | one-line fix, EAGER | 0.69 | 1.98 | 2.68 |
| | FP32 | 13.2 | 11.6 | 16.5 |
| RTX 4090 | zero code | 1.00 | 0.65 | 1.85 |
| | one-line fix | 1.00 | 1.65 | 2.25 |
| | zero code, EAGER | 0.16 | 0.65 | 1.86 |
| | one-line fix, EAGER | 0.59 | 1.66 | 2.25 |
| | FP32 | 7.7 | 7.2 | 11.5 |
| RTX 5060 Ti | zero code | 1.00 | 1.23 | 2.94 |
| | one-line fix | 1.00 | 1.36 | 3.04 |
| | zero code, EAGER | 0.67 | 1.24 | 2.93 |
| | one-line fix, EAGER | 0.86 | 1.37 | 3.04 |
| | FP32 | 12.2 | 18.1 | 22.7 |

TABLE IV

WHAT A USER GETS: SPEEDUP OVER NATIVE FP64. FOR THE GEMMS, ZERO CODE ON THE WALL CLOCK WITH A SYNCHRONISATION AFTER EVERY CALL, AND THE ONE-LINE FIX IN GPU TIME; FOR THE DECOMPOSITION, BOTH ON THE WALL CLOCK. MEDIANS OVER RUNS; ZERO-CODE RUNS VARIED BY UP TO 55 % (RTX 3090, $n = 1024$; SECTION VI-A), AND ONE-LINE-FIX RUNS BY AT MOST 1.1 %.

| GPU | level of effort | square $n = 1024$ | square $n = 4096$ | skinny $n = 256$ | rSVD *exp*, $\ell = 512$, $q = 2$ |
|---|---|---|---|---|---|
| RTX 3090 | zero code | 1.00 | 1.01 | 1.00 | 1.00 |
| | one-line fix | 1.00 | 1.01 | 1.00 | 1.00 |
| | zero code, EAGER | 0.93 | 3.76 | 0.81 | 2.31 |
| | one-line fix, EAGER | 4.10 | 11.12 | 2.68 | 2.68 |
| RTX 4090 | zero code | 0.56 | 2.64 | 0.48 | 1.85 |
| | one-line fix | 4.06 | 10.46 | 2.17 | 2.25 |
| | zero code, EAGER | 0.62 | 2.52 | 0.46 | 1.86 |
| | one-line fix, EAGER | 4.05 | 10.45 | 2.17 | 2.25 |
| RTX 5060 Ti | zero code | 2.40 | 8.92 | 1.91 | 2.94 |
| | one-line fix | 4.95 | 13.24 | 2.93 | 3.04 |
| | zero code, EAGER | 2.42 | 7.97 | 1.92 | 2.93 |
| | one-line fix, EAGER | 4.96 | 13.24 | 2.93 | 3.04 |

of $AQ_z$. The fallback was again silent, and the results stayed at native accuracy.

**Accuracy stays at the level of FP64, which single precision does not reach.** In every FP64 mode, emulated or not, the relative error of the ten largest singular values lay between $2.2 \times 10^{-16}$ and $4.4 \times 10^{-15}$, against $2.2 \times 10^{-16}$ to $3.8 \times 10^{-15}$ for native runs. FP32 gave $1.8 \times 10^{-7}$ to $1.7 \times 10^{-6}$ and TF32 $5.8 \times 10^{-6}$ to $1.1 \times 10^{-4}$. These ranges leave out *poly*, on which every FP64 mode, native included, reached errors of up to $2.3 \times 10^{-7}$ without power iterations, a limit set by the algorithm rather than by the arithmetic. There the emulated errors differed from the native ones by at most 0.11 %, and with $q = 2$ they lay between $2.2 \times 10^{-16}$ and $1.8 \times 10^{-15}$, against $3.3 \times 10^{-16}$ to $1.4 \times 10^{-15}$ for native runs. Where the native residual lay above the rounding floor ($10^{-14}$), the emulated residual was 0.75 to 1.00 times the native one, and below that floor it never exceeded $7.5 \times 10^{-15}$. FP32 was 6.8× to 26× faster than native FP64, so it remains much faster than emulation, but it is not a substitute for FP64 in this workload, whereas emulation is. These results support H4.

### F. What a user gets

Table IV brings the previous subsections together. For each GPU, it gives the speedup that each way of using the emulation delivers at four representative points, from a moderate square product to the full decomposition.

**On the RTX 3090, nothing happens unless the user asks for EAGER.** With the default strategy, both zero code and the one-line fix left every point at native speed. Under EAGER, zero code reached 3.76× at $n = 4096$ and 2.31× on the decomposition but lost on the skinny product and stayed within run-to-run variation of native at $n = 1024$, while the fix with EAGER delivered 4.10×, 11.12×, 2.68×, and 2.68×.

**On the RTX 4090, zero code can be worse than doing nothing.** It ran at 0.56× for $n = 1024$ and at 0.48× for the skinny product, and only the one-line fix turned these into speedups of 4.06× and 2.17×. EAGER added nothing at these points, since the default already emulates them.

**On the RTX 5060 Ti, zero code keeps more of the gain than on the other GPUs.** It reached 2.40×, 8.92×, 1.91×, and 2.94×, against 4.95×, 13.24×, 2.93×, and 3.04× with the fix, and so kept between about half and two thirds of the fix's speedup on the GEMMs and nearly all of it on the decomposition, which follows from the lowest remap rate of the three GPUs.

Taken together, the measurements answer the four hypotheses as follows:

- **H1 holds for the capability, but not for the zero-code opt-in on every GPU.** With the one-line fix, and EAGER on the RTX 3090, emulation ran 14.5× to 18.8× faster than native at its peak and at least ten times more accurately on all three GPUs, although at $n = 12288$ the RTX 5060 Ti ran native in every mode (Section VI-A). Yet the default strategy never emulates on the RTX 3090, and zero code, with a synchronisation after every call, loses part of the gain on the other two, and most of it on the RTX 4090 up to $n = 4096$.
- **H2 holds for the algorithm, and the default strategy guards against it in GPU time.** Emulation lost below a varying dimension of 48 to 192 in our shape families, but none of the 142 configurations that the default emulated, and for which we measured native, ran slower in GPU time, at the price of declining some shapes on which it would have won. On the wall clock, however, zero code still ran 13 of the shapes that the default emulates below native on the RTX 4090 (Section VI-D), so this guard does not extend to the memory penalty.
- **H3 holds only for the widest ranges.** As predicted, a fallback raises no error or warning, and a user who changes no code can see it only in the timing or with a profiler. On *scaled8*, the one matrix on which the fallback set in, every product $AQ_z$ of the power iterations fell back, and with $q = 2$ every mode that emulated kept only 39 % to 49 % of its gain over native, $S - 1$, wherever the decomposition stayed faster than native, while zero code on the RTX 4090 at $\ell = 256$ fell below native. Without power iterations *scaled8* lost at most 1.4 %, and on *scaled4* the speedup stayed within 4 % of that on *exp*, so the loss came with the fallback. The fallback set in, however, only where the range inside the dot products needed at least 107 mantissa bits.
- **H4 holds.** The final answers of the workload stayed at the level of native FP64 in every emulated mode.

## VII. Conclusion

What cuBLAS's automatic FP64 emulation delivers on a consumer GPU depends on three things that the single switch does not reveal: whether the default strategy emulates on that GPU at all, whether the program gives the emulation the memory it needs or synchronises rarely enough to do without it, and whether the inputs stay within the range that the run-time check accepts. The third arose least often, since a fallback took a range inside the dot products that needed at least 107 mantissa bits, or a NaN or infinity among the inputs, but where it arose, on the most badly scaled matrix of the workload, about one in five of the calls that entered emulation fell back, all of them in the power iterations, and more than half of the gain was lost. In all three cases, the call returns success without a warning. The optional counter of mantissa bits exposes a decline and most fallbacks to a program that requests it, but the cost of missing memory shows up only in the timing. Accuracy, by contrast, held wherever cuBLAS chose the precision itself. Across 2624 measurements of single products, every call it emulated with dynamic precision was at least as accurate as native FP64, and inputs beyond the limit fell back to native arithmetic instead of losing accuracy. In the cases and metrics we measured, the risk of switching emulation on was therefore a slower program rather than a wrong answer, and should NVIDIA make emulation the default, users who never opted in would bear that risk without being told.

Our measurements suggest four practical steps for a user who opts in:

- **Give the emulation its memory.** Setting a workspace with `cublasSetWorkspace` removes the zero-code penalty, and in libraries that reset the workspace, such as CuPy, raising the release threshold of the memory pool has the same effect. A program pays the penalty about once per synchronisation, so it matters most when little work separates synchronisations, as in our randomised SVD at $\ell = 256$ on the RTX 4090.
- **Choose the strategy by GPU and shape.** On the RTX 3090 only EAGER emulates, and under EAGER, in our shape families, products whose small dimension lay below 48 to 192 ran slower than native. Because cuBLAS also accepts the strategy for each handle through `cublasSetEmulationStrategy` [8], a program can reserve EAGER for a handle that serves only large products.
- **Confirm that emulation happens.** The counter of Section V-C stays unwritten on a decline and, except at exactly 107 bits, reveals a fallback through its value. This matters most on an 8 GiB card, where a large product may run natively without notice.
- **Fix the precision only when the input range is known.** On well-scaled inputs, 53 fixed bits matched the accuracy of the default at about 1.5 times its speed, but too few bits on a wide range lose accuracy without any warning.

### A. Limitations

Our conclusions are subject to the following limitations:

- **One GPU model per architecture.** Each architecture is represented by a single GPU model. The RTX 4090 is a board modified to 48 GiB of memory, and the RTX 5060 Ti sits in a PCIe Gen3 x8 slot, which may affect wall-clock measurements but not GPU time.
- **A host without clock control.** We could not lock the GPU clocks or reserve the host's processors entirely. No measurement showed hardware throttling. Repeats of FP64 runs without zero code agreed within about 3 % in

both the GEMM benchmark and the workload, and FP32 and TF32 runs of the workload within 19 %, but zero-code runs varied by more than 20 % in 58 of 352 GEMM configurations and by up to 48 % across all settings of the workload, though by at most 16 % in the cells of Table III. The size of the zero-code penalty is therefore specific to our host and to how often a program synchronises, even though its mechanism, and the model that predicts it, are not.

- **One library version.** Our main results concern cuBLAS 13.8.0.4. The default strategy, the precision limit, and the memory behaviour may change in later releases, and our comparison of documented and observed behaviour applies to this version alone.
- **Synthetic inputs and one workload.** All our inputs, including the test matrices of the randomised SVD, are synthetic, and the randomised SVD is our only application, so other applications may combine shapes and value ranges differently.
- **Accuracy from samples and summary measures.** The error of a single product comes from 4096 sampled entries, and the accuracy of the workload from its ten largest singular values and its residual, so neither establishes correctness for every entry, input, or application.
- **A single pass over the decision map.** The decision map recorded each path once, and its timings come from only three calls per shape. The path never changed between repeats in our other experiments, but a single pass cannot rule out variation at individual points, and short timings are least reliable where EAGER and native are close.
- **Break-even against cuBLAS's own native kernels.** Our speedups divide by the time of the native kernel that cuBLAS selects, which does not vary smoothly with the dimension, so a different native baseline would move the break-even points.
- **Explanations from structure rather than measurement.** Several explanations rest on the structure of the schemes or on the documentation rather than on a direct measurement. These include why the speedup grows with $n$, why small dimensions carry a fixed cost, why longer dot products need fewer bits, which of the two products of the power iterations falls back in the workload, and why CuPy discards a user's workspace.

### B. Future work

Several directions follow from these limitations. Datacentre GPUs with strong native FP64, such as the A100 and H100, would show how the default strategy behaves when emulation has less to gain. Newer releases of cuBLAS should be checked for changes to the memory behaviour, to the decisions on Ampere, and to the precision limit. A breakdown of kernel time by phase across sizes would test our structural explanations, and tracing the workload call by call would confirm which product falls back. Finally, applications whose matrix products run inside other libraries, such as the factorisations of cuSOLVER, would show how far the opt-in reaches in practice.

## Acknowledgment

We thank Sherman Chann (https://152334h.github.io/about/) for access to his GPU server, on which all measurements in this paper were made.

## AI Use Declaration

This paper was written with assistance from Anthropic's Opus 5 series models, to help with research, fact-checking, editing, and proofreading.

## References

[1] A. Woodie, "Genesis mission will lean heavily on Ozaki scheme for FP64 capability," HPCwire, Feb. 17, 2026, accessed: Nov. 8, 2026. [Online]. Available: https://www.hpcwire.com/2026/02/17/genesis-mission-will-lean-heavily-on-ozaki-scheme-for-fp64-capability/

[2] Y. Uchino, K. Ozaki, and T. Imamura, "Performance enhancement of the Ozaki scheme on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 39, no. 3, pp. 462–476, 2025.

[3] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications," *Numerical Algorithms*, vol. 59, pp. 95–118, 2012.

[4] K. Ozaki, Y. Uchino, and T. Imamura, "Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique," *The International Journal of High Performance Computing Applications*, 2026, online first; also arXiv:2504.08009.

[5] S. Matsuoka, "FP8 is all you need (Part 1): Debunking hardware FP64 as the HPC holy grail," arXiv preprint arXiv:2606.06510v4, 2026.

[6] C. Brower, J. Gunnels, and G. Lopez, "Unlocking tensor core performance with floating point emulation in cuBLAS," NVIDIA Technical Blog, Oct. 24, 2025, accessed: Nov. 8, 2026. [Online]. Available: https://developer.nvidia.com/blog/unlocking-tensor-core-performance-with-floating-point-emulation-in-cublas/

[7] A. Schwarz *et al.*, "Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme," in *Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia 2026)*. Osaka, Japan: ACM, 2026, pp. 91–101.

[8] NVIDIA Corporation, *cuBLAS Library, Release 13.4*, NVIDIA Corporation, 2026, CUDA Toolkit 13.4 Update 1 archive, accessed: Nov. 8, 2026. [Online]. Available: https://docs.nvidia.com/cuda/archive/13.4.2/pdf/CUBLAS_Library.pdf

[9] S. Rodriguez, "Floating point emulation in NVIDIA math libraries," Talk at the CERN workshop "Optimizing Floating Point Precision", Geneva, Switzerland, Jul. 2025, slides, accessed: Nov. 8, 2026. [Online]. Available: https://indico.cern.ch/event/1538409/contributions/6521976/attachments/3096181/5485165/cern-talk.pdf

[10] A. Woodie, "AMD hints at big FP64 increases in MI430X GPU as Ozaki underwhelms," HPCwire, Mar. 13, 2026, accessed: Nov. 8, 2026. [Online]. Available: https://www.hpcwire.com/2026/03/13/amd-hints-at-big-fp64-increases-in-mi430x-gpu-as-ozaki-underwelms/

[11] T. Mann, "Nvidia leans on emulation to squeeze more HPC oomph from AI chips in race against AMD," The Register, Jan. 18, 2026, accessed: Nov. 8, 2026. [Online]. Available: https://www.theregister.com/2026/01/18/nvidia_fp64_emulation/

[12] Y. Uchino, K. Ozaki, and T. Imamura, "High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*. St. Louis, MO, USA: ACM, 2025, pp. 1824–1831.

[13] D. Lu, A. Maeder, M. Luisier, and A. N. Ziogas, "EmuGEMM: Fused tensor core kernels for precision emulation in matrix multiplication," arXiv preprint arXiv:2606.25453, 2026, to appear in SC26.

[14] soy-tuber, "FP64 emulation on a GeForce RTX 5090," GitHub repository README, 2026, accessed: Nov. 8, 2026. [Online]. Available: https://github.com/soy-tuber/fp64-emulation-rtx5090

[15] s-sasaki-earthsea-wizard, "Add CUDA 12 vs 13.4 Ozaki-scheme FP64 benchmark suite with RTX 5080 results," GitHub pull request #1, vrc-ozaki-schema-LT, Oct. 5, 2026, accessed: Nov. 8, 2026. [Online]. Available: https://github.com/s-sasaki-earthsea-wizard/vrc-ozaki-schema-LT/pull/1

[16] H. Ootomo, K. Ozaki, and R. Yokota, "DGEMM on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.

[17] D. Mukunoki, "DGEMM without FP64 arithmetic – using FP64 emulation and FP8 tensor cores with Ozaki scheme," arXiv preprint arXiv:2508.00441, 2025.

[18] Y. Uchino, K. Ozaki, and T. Imamura, "Double-precision matrix multiplication emulation via Ozaki-II scheme with FP8 quantization," arXiv preprint arXiv:2603.10634, 2026.

[19] S. Kawakami and D. Takahashi, "Improved scaling for fast mode of Ozaki scheme II," arXiv preprint arXiv:2606.29129, 2026.

[20] A. Abdelfattah, J. Dongarra, M. Fasi, M. Mikaitis, and F. Tisseur, "Analysis of floating-point matrix multiplication computed via integer arithmetic," arXiv preprint arXiv:2506.11277, 2025.

[21] Y. Uchino, K. Ozaki, and T. Imamura, "Error analysis of matrix multiplication emulation using Ozaki-II scheme," arXiv preprint arXiv:2602.02549, 2026.

[22] RIKEN Center for Computational Science, "GEMMul8: GEMM emulation and its extension to BLAS-like matrix operations using INT8/FP8 matrix engines," GitHub repository, 2026, accessed: Nov. 8, 2026. [Online]. Available: https://github.com/RIKEN-RCCS/GEMMul8

[23] J. Demmel *et al.*, "How to grade the accuracy of the BLAS," arXiv preprint arXiv:2609.12307, 2026.

[24] H. Liu, J. Li, and Y. Wang, "A pilot study on tunable precision emulation via automatic BLAS offloading," arXiv preprint arXiv:2503.22875, 2025.

[25] H. Liu, J. Li, Y. Wang, N. K. Nepal, and Y. Wang, "A precision emulation approach to the GPU acceleration of ab initio electronic structure calculations," arXiv preprint arXiv:2603.29975, 2026.

[26] P. Luszczek *et al.*, "Performance and numerical aspects of decompositional factorizations with FP64 floating-point emulation in INT8," in *2025 IEEE High Performance Extreme Computing Conference (HPEC)*. Wakefield, MA, USA: IEEE, 2025, pp. 1–7.

[27] H. Ootomo and R. Yokota, "Mixed-precision random projection for RandNLA on tensor cores," in *Proceedings of the Platform for Advanced Scientific Computing Conference (PASC '23)*. Davos, Switzerland: ACM, 2023, pp. 1–11.

[28] M. Baboulin, S. Donfack, O. Kaya, T. Mary, and M. Robeyns, "Mixed precision randomized low-rank approximation with GPU tensor cores," in *Euro-Par 2024: Parallel Processing*. Springer, 2024, pp. 31–44.

[29] NVIDIA Corporation, *CUDA Toolkit 13.4 Update 1 Release Notes*, NVIDIA Corporation, Sep. 14, 2026, accessed: Nov. 8, 2026. [Online]. Available: https://docs.nvidia.com/cuda/archive/13.4.2/pdf/CUDA_Toolkit_Release_Notes.pdf

[30] T. Ogita, S. M. Rump, and S. Oishi, "Accurate sum and dot product," *SIAM Journal on Scientific Computing*, vol. 26, no. 6, pp. 1955–1988, 2005.

[31] N. Halko, P.-G. Martinsson, and J. A. Tropp, "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions," *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.

[32] J. A. Tropp, A. Yurtsever, M. Udell, and V. Cevher, "Practical sketching algorithms for low-rank matrix approximation," *SIAM Journal on Matrix Analysis and Applications*, vol. 38, no. 4, pp. 1454–1485, 2017.